1️⃣ Naive Bayes

What is it?

A classification algorithm based on **Bayes Theorem** and the assumption that all features are **independent** of each other. That independence assumption is the "naive" part.

Bayes Theorem — Simple Explanation

It calculates the probability of something being true given some evidence.

$$P(\text{Disease} \mid \text{Symptoms}) = \frac{P(\text{Symptoms} \mid \text{Disease}) \times P(\text{Disease})}{P(\text{Symptoms})}$$

In simple words — given these symptoms, what is the probability the patient has this disease?

How it works

- Looks at each feature independently
- Calculates probability of each class for given features
- Assigns the class with highest probability

Real Example — Spam Detection

Email has words — "free", "win", "prize", "click" Model calculates:

- Probability this is spam given these words
- Probability this is not spam given these words
- Assigns whichever is higher

Types of Naive Bayes

- **Gaussian Naive Bayes** — for continuous numerical data
- **Multinomial Naive Bayes** — for text data, word counts
- **Bernoulli Naive Bayes** — for binary data, yes or no features

Best for

- Text classification — spam detection, sentiment analysis
- Very fast training and prediction
- Works well with small datasets
- Real time prediction

Weakness

- Assumption of feature independence is rarely true in real data
- If a feature value never appeared in training — probability becomes zero
- Not great for complex relationships between features

2️⃣ Support Vector Machine — SVM

What is it?

SVM finds the **optimal boundary** called a hyperplane that separates classes with the **maximum margin** between them.

Key Concepts

Hyperplane: The decision boundary that separates classes. In 2D it is a line. In 3D it is a plane. In higher dimensions it is a hyperplane.

Support Vectors: The data points closest to the hyperplane. These are the most important points — they define the boundary. If you remove other points the hyperplane stays the same.

Margin: The distance between the hyperplane and the nearest data points from each class. SVM maximizes this margin. Larger margin means better generalization.

Hard Margin vs Soft Margin:

- Hard margin — no misclassification allowed. Works only when data is perfectly separable.
- Soft margin — allows some misclassification. More practical for real world data.

Kernel Trick — Most Important Concept

When data is not linearly separable in current dimensions — kernel trick maps data to higher dimensions where it becomes separable.

Think of it like this — two groups of points mixed together on a flat surface. If you lift some points up into 3D space — they become separable by a flat plane.

Common Kernels:

- **Linear** — for linearly separable data
- **RBF — Radial Basis Function** — most commonly used, handles non-linear data
- **Polynomial** — for polynomial decision boundaries

Best for

- High dimensional data — many features
- Text classification
- Image classification
- When classes are clearly separable
- Small to medium datasets

Weakness

- Slow on large datasets
- Choosing right kernel and parameters is tricky
- Hard to interpret — cannot easily explain why it made a decision
- Feature scaling is mandatory

3️⃣ XGBoost — Extreme Gradient Boosting

What is it?

XGBoost is an advanced **ensemble algorithm** that builds trees **sequentially** where each new tree corrects the errors of the previous trees. This technique is called **Gradient Boosting.**

Boosting vs Bagging — Key Difference

Bagging — Random Forest:

- Builds many trees **in parallel** independently
- Each tree trained on random subset of data
- Final answer is majority vote
- Reduces variance — prevents overfitting

Boosting — XGBoost:

- Builds trees **sequentially** one after another
- Each new tree focuses on mistakes of previous tree
- Final answer is weighted sum of all trees
- Reduces bias — corrects errors progressively

How XGBoost Works Step by Step

1. Start with a simple prediction — mean of target
2. Calculate errors — difference between prediction and actual
3. Build a tree to predict these errors
4. Add this tree to the model
5. Recalculate remaining errors
6. Build another tree to predict new errors
7. Repeat until errors are minimized or maximum trees reached

Why XGBoost is so Powerful

- Built-in regularization — prevents overfitting
- Handles missing values automatically
- Very fast — uses parallel processing
- Works with numerical and categorical data
- Consistently wins Kaggle competitions

Key Hyperparameters

- **n_estimators** — number of trees
- **learning_rate** — how much each tree contributes

- **max_depth** — depth of each tree
- **subsample** — fraction of data used per tree

Best for

- Structured tabular data
- Classification and regression both
- When you need highest accuracy
- Kaggle competitions and real projects

Weakness

- Many hyperparameters to tune
- Can overfit if not tuned carefully
- Less interpretable than single decision tree
- Slower than simple models

🔽 PCA — Principal Component Analysis

What is it?

PCA is a **dimensionality reduction** technique that reduces the number of features while keeping the most important information.

Why do we need it?

Real datasets can have hundreds or thousands of features. Too many features causes:

- Slow training
- Overfitting
- Hard to visualize
- Curse of dimensionality — model performance drops with too many features

How it Works — Simple Explanation

Think of it like taking a 3D object and finding the best 2D shadow of it that preserves the most information.

PCA finds new directions in data called **Principal Components** that capture the maximum variance.

Step by step:

1. Standardize the data
2. Calculate covariance matrix — how features relate to each other
3. Find eigenvectors and eigenvalues
4. Sort by eigenvalue — highest variance first
5. Select top N components

6. Transform data into new dimensions

Key Concepts

Principal Components: New features created by PCA. They are combinations of original features. First component captures most variance. Second captures second most. And so on.

Explained Variance: How much information each component captures. If first 2 components explain 95% variance — you can reduce to 2 dimensions with minimal information loss.

Variance: Spread of data. PCA tries to find directions where data spreads the most.

Real Example

Dataset has 50 features. PCA reduces to 10 components that explain 95% of variance. Now model trains on 10 features instead of 50 — faster and less overfitting.

When to Use PCA

- Too many features — high dimensional data
- Features are correlated with each other
- Want to visualize high dimensional data in 2D or 3D
- Want to speed up training
- Want to reduce noise in data

Weakness

- Components are hard to interpret — not original features anymore
- Information loss — even if small
- Only captures linear relationships
- Must standardize data before applying PCA

PCA vs Feature Selection

PCA	Feature Selection
Creates new features	Keeps original features
Combines all features	Removes some features
Hard to interpret	Easy to interpret
No information loss goal	Removes least important

🗣️ **Interview Answers — Practice These**

What is Naive Bayes?

"Naive Bayes is a probabilistic classification algorithm based on Bayes theorem. It assumes all features are independent of each other — which is the naive assumption. Despite this simplification it works very well for text classification tasks like spam detection and sentiment analysis because it is fast and works well with small datasets."

What is SVM?

"SVM finds the optimal hyperplane that separates classes with maximum margin. The data points closest to the hyperplane are called support vectors. When data is not linearly separable, SVM uses the kernel trick to map data to higher dimensions where it becomes separable. RBF kernel is the most commonly used."

What is the difference between Bagging and Boosting?

"Bagging builds trees in parallel independently on random subsets of data and combines by majority vote — Random Forest is an example. Boosting builds trees sequentially where each tree corrects errors of the previous one — XGBoost is an example. Bagging reduces variance while boosting reduces bias."

What is PCA?

"PCA is a dimensionality reduction technique that reduces the number of features while preserving maximum information. It finds new directions called principal components that capture the most variance in data. We use it when we have too many features, when features are correlated, or when we want to visualize high dimensional data."

✅ **Summary Table — All Algorithms**

Algorithm	Type	Best for	Key weakness
Linear Regression	Regression	Continuous prediction	Only linear relationships
Logistic Regression	Classification	Binary classification	Linear boundary only
KNN	Classification	Small datasets	Slow on large data
Decision Tree	Both	Interpretable models	Overfits easily
Random Forest	Both	General purpose	Less interpretable
SVM	Classification	High dimensional data	Slow on large data
Naive Bayes	Classification	Text classification	Independence assumption
XGBoost	Both	Highest accuracy	Many parameters
K-Means	Clustering	Large datasets	Need to specify K
DBSCAN	Clustering	Irregular shapes	Parameter sensitive
PCA	Dimensionality Reduction	Too many features	Hard to interpret

---

Now tell me — Stage 2 or Stage 3 notes next? 💪